

LLD Case Study 9: Snake and Ladders Game

Target Patterns: Factory Pattern, Strategy Pattern, State Machine

Key Engineering Focus: Modular board layout, jump cell behaviors (Snakes vs Ladders), deterministic dice strategies, and multi-player game loops.

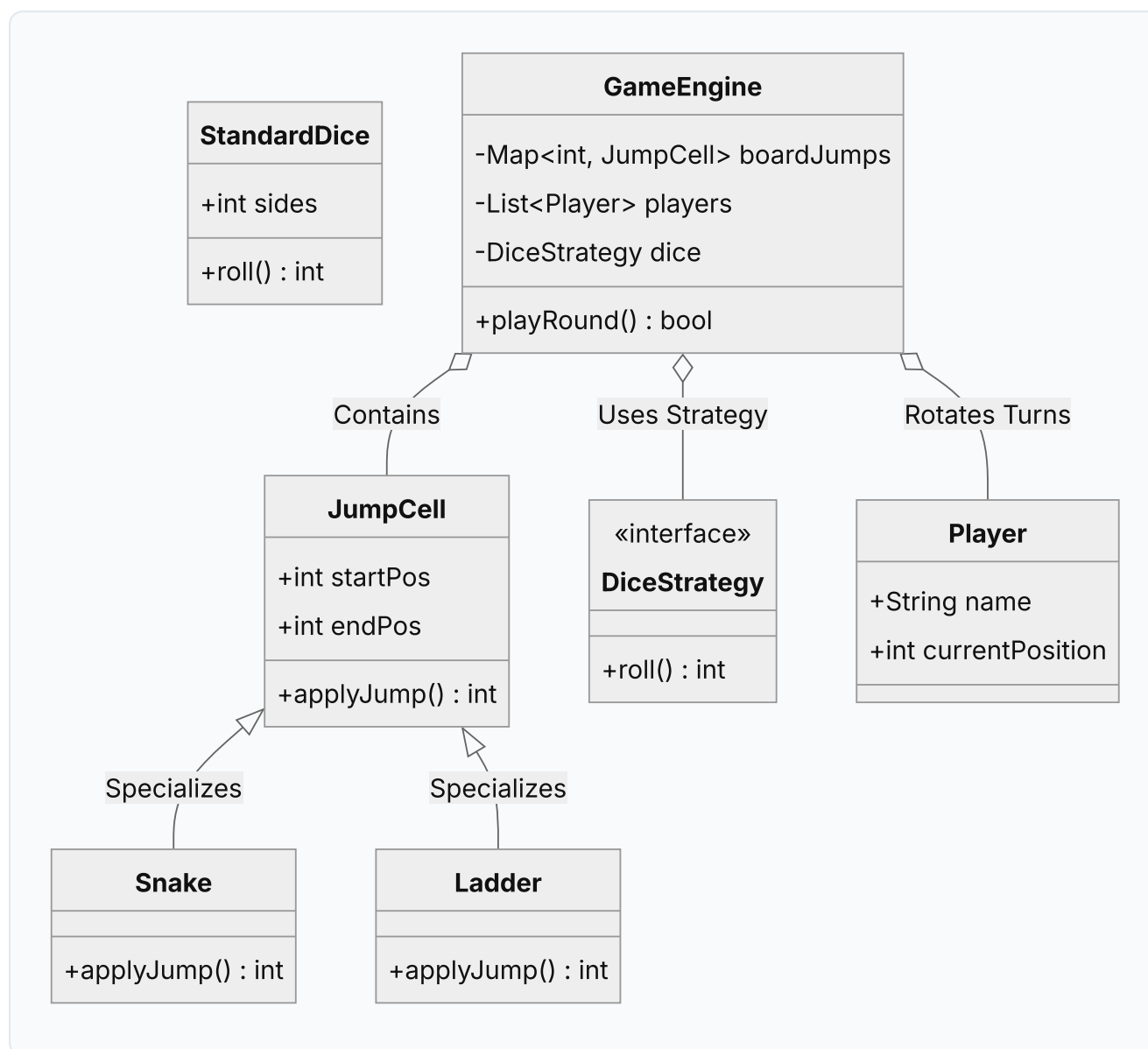
1. Problem Statement & Functional Requirements

Design a modular, extensible Snake and Ladders board game.

Requirements:

1. **Configurable Board:** Board with N cells (default 1 to 100).
 2. **Jump Entities:** Snakes (drops player back) and Ladders (propels player forward).
 3. **Dice Strategy (Strategy Pattern):** Pluggable dice mechanics (Single 6-sided dice, Biased/Rigged dice for testing, Dual dice).
 4. **Game State Engine (State Pattern):** Turns rotate in round-robin fashion until a player lands exactly on cell 100 to win.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

Rules the implementation has to get right

Case	What the code does	Note
Roll overshoots 100	Player stays put; turn passes	Matches the "exact finish" house rule; some variants bounce back instead
Landing on a snake head or ladder base	One jump is applied	A ladder whose top is a snake head is not chained: the player stops on the snake head and is not bitten
Two jumps share the same start cell	The second <code>add_*</code> silently overwrites the first	Reject at board-build time
Jump starting on cell 100	Winner check happens after the jump, so a snake on 100 would undo a win	Reject at board-build time
Game already won	<code>take_turn()</code> returns <code>True</code> without mutating anything	Idempotent end state

The first three are board-validation problems, not game-loop problems. Validating once when the board is built is cheaper and safer than adding checks to every turn.

Tests with rigged dice

`DeterministicDice` exists so that a game is a pure function of the roll sequence. This block extends the implementation above.

Extensions interviewers ask for

1. **Multiple dice, or "roll again on a six":** only `DiceStrategy` and the turn loop change; keep a `Turn` object that can request an extra roll.
2. **Board of arbitrary size:** pass `size` into the game instead of the class constant; `validate_board` already takes it.
3. **Persist and resume a game:** the state is the player positions, the queue order and the dice sequence position; serialise those three.
4. **Simulate a million games for statistics:** inject a seeded `random.Random` into `StandardDice` so runs are reproducible, and expect the average game to take about 40 turns on the classic board (measure it, it depends on the board).

Follow-up questions

- *Why is the dice a strategy rather than a call to `random`?* Testability: randomness is the only non-determinism in the game, so injecting it makes every test exact.
- *What makes a board unwinnable?* Snakes and ladders that form a cycle, or a snake on the last reachable cells combined with exact-finish rules; `validate_board` catches the first kind, a reachability search (BFS over cells 1 to 100 with dice outcomes 1 to 6) catches both.